OpticOS: design specification

Version 0.3, 13 November 2026. The six design decisions in §12 were agreed the same day. v0.3 rebalances the §8.2 budget after phase 3. Project, ROM and future repo name: **OpticOS**. Boot banner: `OpticOS (c) 2026 Optica Technology`.

This document covers design and planning only. The Z80 snippets in it are there to show technique. They are not the implementation.

---

1. Goals and non-goals

Goals

1. **A drop-in 16,384-byte ROM image** for the 48K (and 16K) Spectrum that boots on real hardware, in Fuse and in our headless rig.
2. **OpticBASIC, a modern structured BASIC with no line numbers.** It has named procedures and functions with parameters and `LOCAL`, block `IF/ELSEIF/ELSE/ENDIF`, `WHILE`, `REPEAT`, `FOR`, `CASE`, labels in place of line-number `GOTO`, and `TRY/CATCH` error handling.
3. **64-column text as the default** (a condensed 4-pixel font in the spirit of Tasword Two) for PRINT and the editor, with a `MODE 32` switch.
4. **A full-screen editor**. You type keywords letter by letter and a tokeniser converts them. There is no single-key keyword entry.
5. **Tape compatibility.** OpticOS uses the standard ROM tape encoding (pilot, sync, bit timings, flag and parity bytes, the 17-byte header layout), so ordinary TAP and TZX files load and OpticOS saves load on other tools.
6. **Machine-code compatibility where it is cheap.** RST vectors, IM 1 at `$0038`, NMI at `$0066`, LD-BYTES at `$0556`, SA-BYTES at `$04C2`, BEEPER at `$03B5` and a handful of others keep their addresses and register contracts.
7. **Speed.** The target is at least 3× Sinclair BASIC on integer loops, using a 16-bit integer fast path and pre-resolved control flow.
8. **Clean room.** OpticOS contains no Sinclair, Amstrad or Tasword code or artwork (see §1.3).

Non-goals

- Running existing Sinclair BASIC programs. The exception is the small **legacy loader** subset that tape loaders use (§3.4).
- Emulating the RST `$28` calculator byte-code interface.
- 128K paging, AY sound, the ZX Printer, Interface 1 and Microdrive. The `$0008` hook stays, so a later shadow ROM could still page in.
- An inline assembler like BBC BASIC's. It is too expensive in 16K. `CALL` and `USR` cover machine code.
- `MERGE`, `DATA` arrays on tape, `DEF FN` in the Sinclair style, `RENUM` and `AUTO`. None of these make sense without line numbers.

1.3 Clean-room rules

- Nathan's `48.rom` (in `/workspace/ultimate/roms/`) is a **behavioural reference only**. We may run it in an emulator to measure register contracts, timings and outputs at the entry points we keep. We do not disassemble it into OpticOS source, and we do not paste its bytes or routines.
- The ROM image, the Ultimate snapshots and the scans are **never committed or published**. `roms/` stays in `.gitignore`, as in the Mire Mare repo.

- Public facts are fair to use: the tape signal format, the header layout, system-variable addresses, token numbers and published entry-point contracts.
- A `similarity_check.py` gate fails the build if `opticos.rom` shares any run of 12 or more identical bytes with `48.rom`. Font areas and runs of filler bytes are excluded from the check.
- **The font is original artwork.** Tasword Two (Tasman Software, 1983) inspired the idea of a 64-column font. We draw our own glyphs on a 3×6 grid, in an ASCII-art source file, without consulting Tasword's or Sinclair's bitmaps.

---

2. Hardware compatibility

Item	Behaviour OpticOS relies on
CPU	Z80A at 3.5 MHz. 69,888 T-states per frame on the 48K. Contended memory at `$4000–$7FFF`.
ULA port `$FE` (any even port)	Writing sets the border (bits 0–2), MIC (bit 3) and EAR/beeper (bit 4). Reading gives keyboard half-rows (high byte selects the row, bits 0–4 active low) and EAR in (bit 6).
Keyboard matrix	8 half-rows × 5 keys. Row-select bytes are `$FE, $FD, $FB, $F7, $EF, $DF, $BF, $7F` (CAPS–V, A–G, Q–T, 1–5, 0–6, P–Y, ENTER–H, SPACE–B).
Kempston	Port `$1F`, active high: bit 0 right, 1 left, 2 down, 3 up, 4 fire. Read by the `JOY` function only. It is never auto-detected, because a floating bus reads `$FF`.
IM 1	Set at boot. The 50 Hz frame interrupt arrives at `$0038`.
NMI	`$0066`. If `NMIADD` (`$5CB0`) is non-zero, jump to it. Otherwise `RETN`.
Screen	Bitmap `$4000–$57FF` in the usual third/row/line interleave. Attributes `$5800–$5AFF`.
RAM size	The boot memory test finds 16K (`P_RAMT=$7FFF`) or 48K (`$FFFF`).

Interrupt routine (`$0038`)

The routine does not use IY. It addresses the system variables directly. A game that repurposes IY and leaves IM 1 running crashes the Sinclair ROM, but it survives OpticOS. On each interrupt the routine:

1. Pushes AF, BC, DE and HL.
2. Increments the 24-bit `FRAMES` counter (`$5C78`).
3. Scans the keyboard into `KSTATE`-style debounce slots. It writes `LAST_K` (`$5C08`) and sets bit 5 of `FLAGS` (`$5C3B`), with `REPDEL` and `REPPER` auto-repeat.
4. Toggles the editor cursor every 16 frames when the editor is active.
5. Pops the registers, then `EI` and `RET`.

The budget is about 1,000 T-states on a typical frame and no more than 1,600.

---

3. Compatibility policy

3.1 Tape format (kept exactly)

Element	Value
Pilot	2168 T per half-pulse. 8063 pulses for a header (flag `$00`), 3223 for data (`$FF`).
Sync	667 T, then 735 T.
Bits	0 = two 855 T half-pulses. 1 = two 1710 T half-pulses. MSB first.
Block	Flag byte, then data, then an XOR parity byte.
Header	17 bytes: type (0 program, 1 number array, 2 char array, 3 code), 10-char name, length, param 1, param 2.

OpticOS programs are saved as **type 0** headers, so tape catalogues still say "Program:". The data block begins with the 4-byte signature `"OB" $01 $00`. On LOAD, a type-0 block without this signature is a Sinclair BASIC program and is handed to the legacy loader (§3.4).

3.2 Fixed entry points

Each one is either the real routine or a 3-byte `JP` stub at the original address. Its register contract is documented and tested.

Address	Name	Contract kept	Why	Cost
`$0000`	RESET	Cold start	Universal	in init
`$0008`	ERROR (RST 8)	Next byte is the error code	Some games and Interface 1	0
`$0010`	PRINT-A (RST 16)	Print A to the current channel	Very common in loaders and type-in code	0
`$0018/$0020`	GET/NEXT-CHAR	OpticOS's own interpreter fetch. Not guaranteed.	Vector slots	0
`$0028`	RST 40	**Not** the Sinclair calculator. Raises error "Calculator not supported".		0
`$0030`	RST 48	Workspace allocate (OpticOS's own)		0
`$0038`	IM 1 ISR	FRAMES and keyboard	Universal	in ISR
`$0066`	NMI	As §2 (the Sinclair bug is fixed)		4
`$03B5`	BEEPER	DE = cycles, HL = period, as documented	Many games use it for sound	stub
`$04C2`	SA-BYTES	IX = start, DE = length, A = flag	Savers and utilities	real routine
`$0556`	LD-BYTES	IX, DE, A = flag, carry set = load, carry clear = verify. Returns carry set on success.	**Most important.** Called by countless loaders.	real routine
`$05E3/$05E7`	LD-EDGE-2/1	B = timing counter in and out, carry clear = BREAK or time-out	Used by many custom and turbo loaders	real routine
`$096F`	`RET` byte	Makes the `POKE 23739,111` "hide Bytes: names" trick harmless	Seen in both Mire Mare and Bounce Ball loaders	1
`$0D6B`	CLS	Clear the screen with ATTR_P	Cheap and common	stub
`$0DAF`	CL-ALL	Same as CLS, and resets the print position	Cheap	stub
`$1601`	CHAN-OPEN	A = stream (−3..15)	RST 16 users	stub
`$203C`	PR-STRING	DE = address, BC = length	Common	stub
`$22AA`	PIXEL-ADD	B = y, C = x, gives HL = address, A = bit	Some games	stub
`$2D2B/$2DA2`	STACK-BC / FP-TO-BC	Not provided. The calculator stack is not emulated (§12).		0

The LD-EDGE timing is matched by black-box measurement: we feed pulses of known length and compare the returned B with the reference ROM in the rig. We do not read its code. The per-iteration loop time and the starting B values in LD-BYTES must match closely, so that turbo loaders which call `$05E7` with their own thresholds still work.

3.3 What breaks

Breaks	Impact	Mitigation
Existing Sinclair BASIC programs	Type-ins and BASIC games	None. The legacy loader runs only the loader subset.
Deep ROM calls (the calculator, `$0C0A` PO-MSG, `$1C82`-type syntax routines, mid-routine jumps)	A minority of games, and many utilities	Stubs for the top dozen (above). The rest are documented as unsupported.
RST `$28` calculator byte-code	A few games and demos	Raise an error instead of crashing

Breaks	Impact	Mitigation
Sinclair 8×8 font at `$3D00`	Games that print with the ROM font, or set `CHARS=$3C00` and draw their own text	Accepted: it breaks. `MODE 32` uses pixel-doubled glyphs. There is no 8×8 font at `$3D00` for now; it may be revisited in Phase 5 if the margin allows (§12).
ROM checksums, or using ROM bytes as random data or sound	Copy protection and rare effects	None (different but harmless)
Sinclair's 32-column print layout	Loaders that `PRINT AT` assuming 32 columns, through the legacy shim	The legacy shim forces `MODE 32` while it runs
ZX Printer (`LPRINT`/`COPY`)	Negligible	Dropped

3.4 Legacy loader shim

Almost every commercial tape starts with a short BASIC loader. Both of our own test tapes do. Here are the Bounce Ball and Mire Mare loaders, decoded from the TAPs:

```
10 BORDER 0: CLEAR 32767: LOAD ""CODE : RANDOMIZE USR 32768          (Bounce Ball)
10 BORDER 0: PAPER 0: INK 0: CLEAR 24575: LOAD ""SCREEN$ :
   POKE 23739,111: LOAD ""CODE : RANDOMIZE USR 24576                 (Mire Mare)
```

The shim is a very small interpreter for Sinclair-tokenised programs. It loads the program at `$5CCB`, as the original does, because loaders often embed machine code at fixed addresses or `PEEK 23635`. It then interprets only these:

- `BORDER`, `PAPER`, `INK`, `BRIGHT`, `CLS`, `CLEAR n`, `POKE`, `RANDOMIZE USR`, `LOAD "" [CODE|SCREEN$] [a[,b]]`, `PRINT` with literals and `AT`, `REM`, `GO TO`, `RUN` and `:`
- Expressions using `USR`, `PEEK`, `+ - *`, `VAL "…"` and numbers. Numbers are taken from the hidden 5-byte form after each literal (the `$0E` marker), so no ASCII number parsing is needed.

Anything else stops with "Legacy BASIC: unsupported statement". The cost is about 650 bytes. Sinclair's token numbers for these keywords are a published interface, so recognising them is fine.

3.5 Policy (agreed)

Keep the tape format and the entry-point table above, put a `RET` at `$096F`, and include the legacy shim. Do not emulate RST `$28` or the calculator stack. There is no 8×8 ROM font for now; it may be revisited in Phase 5 if the margin allows (§12).

---

4. Language definition (OpticBASIC)

4.1 Lexical rules

- **Keywords** are case-insensitive. LIST shows them in upper case. Identifiers are case-insensitive and folded to lower case.
- **Identifiers** start with a letter, followed by letters, digits or _. Up to 32 characters, all significant. The suffix `%` means a 16-bit integer and `$` means a string. With no suffix, the variable is a number held as a 5-byte float with the small-integer form.
- **Literals**: `123`, `1.5E-3`, hex `$C000`, binary `%10110001`, and strings in double quotes. Write a quote inside a string as `""`.
- **Comments** start with `REM` or `'` and run to the end of the line.
- **Statements** are one per line. `:` is allowed as a separator but is not needed.

- **Static integers** `A%–Z%` live at fixed RAM addresses, with no lookup, BBC-style. They are the fastest variables.

4.2 Data types

Type	Storage	Range or notes
Integer `x%`	2 bytes	−32768..32767. Overflow raises error 6.
Number `x`	5 bytes	Sinclair-layout float: exponent byte plus a 4-byte mantissa, about 9.5 significant digits, ±1.7E38. Integral values in −65535..65535 use the **small-integer form** `00 sign lo hi 00`, and the arithmetic fast path works on that form directly.
String `s$`	3-byte descriptor (length, pointer) plus heap bytes	0–255 characters
Arrays	`DIM a(9)`, `b%(3,3)`, `n$(20)`	Up to 3 dimensions, base 0, so `DIM a(9)` gives 10 elements. Each string array element is its own heap string.
Boolean	Integer	TRUE = −1, FALSE = 0. `AND`, `OR` and `XOR` are bitwise on integers.

4.3 Operator precedence (highest first)

Level	Operators	Associativity
1	`( )`, function calls, array subscripts	
2	`^`	right
3	unary `-`, unary `+`, `NOT`	right
4	`* / DIV MOD`	left
5	`+ -` (+ also joins strings)	left
6	`<< >>` (integer shifts)	left
7	`= <> < > <= >=`	left (non-chaining)
8	`AND`	left
9	`OR XOR`	left

`/` always returns a float. `DIV` is integer division.

4.4 Statements

Structure and procedures

```
DEF PROC name[(p1, p2$, p3%)]  … ENDPROC
DEF FN name[(params)]          … RETURN expr … ENDFN
DEF FN name(params) = expr                  ' one-line form
LOCAL v, w$, k%, arr()                      ' first lines of a PROC or FN only
name [args]   or   name(args)               ' call a PROC (bare name)
LABEL name    /   GOTO name                 ' escape hatch, same PROC only
END   STOP   REM   '
```

Parameters are passed by value. Arrays are passed by reference using `a()`. Scoping is **dynamic, BBC-style**: `LOCAL` and parameters save the old value on the control stack and restore it at `ENDPROC`, `ENDFN` or error unwind. Recursion is supported.

Control flow

```
IF c THEN stmt [ELSE stmt]                ' one-line form
IF c                                      ' block form (nothing after the condition)
  …
ELSEIF c
  …
ELSE
  …
ENDIF
WHILE c … WEND
REPEAT … UNTIL c
FOR v = a TO b [STEP s] … NEXT [v]
EXIT FOR | EXIT WHILE | EXIT REPEAT
CASE expr OF
  WHEN v1, v2   …
  WHEN v3       …
  OTHERWISE     …
ENDCASE
TRY … CATCH … ENDTRY
ERROR n [, msg$]                          ' raise an error
```

Data

`[LET] v = expr`, `DIM`, `SWAP a, b`, `DATA`, `READ`, `RESTORE [label]`, `CLEAR [ramtop]`, `RANDOMIZE [n]`

Screen and text

Statement	Notes
`CLS`	Clears with ATTR_P
`PRINT [AT r,c;] [TAB c;] items`	`;` means no gap. `,` jumps to the next 16-column zone. `'` starts a new line.
`INPUT ["prompt";] v [, v…]`	One-line editor on the bottom row
`INK n`, `PAPER n`, `FLASH 0/1`, `BRIGHT 0/1`, `INVERSE 0/1`, `OVER 0/1`	Can also appear inline in PRINT. `8` means transparent.
`BORDER n`	
`MODE 64` / `MODE 32`	Switches the text width. Also clears the screen and homes the cursor.
`UDG c$, b0, …, b7`	Defines a UDG. `UDG("A")` returns its address.

Columns in `AT` and `TAB` are **0–63 in MODE 64** and 0–31 in MODE 32. Rows are 0–23. Row 23 is available to PRINT as long as no `INPUT` is pending.

Graphics (256×192 pixels, origin bottom-left; y = 0–191, since OpticOS keeps no permanent lower screen; INPUT temporarily uses row 23)

`PLOT x,y`, `DRAW dx,dy` (relative, Bresenham), `DRAW TO x,y` (absolute), `CIRCLE x,y,r` (integer midpoint). There are no Sinclair arcs: `DRAW dx,dy,angle` is a non-goal.

Sound

`BEEP dur, pitch` works as on Sinclair, with pitch in semitones from middle C. `SOUND period, cycles` drives the beeper directly with integers.

Machine and I/O

`POKE a,b`, `DPOKE a,w`, `OUT port,b`, `CALL addr [,a%,bc%,de%,hl%]` and `WAIT n`. `WAIT n` halts for n frames.

Tape

```
SAVE "name"                    ' program (+ AUTO to autostart)
SAVE "name" AUTO [label]
SAVE "name" CODE start, len
SAVE "name" SCREEN$
LOAD "name" | LOAD ""          ' program (OpticOS or legacy)
LOAD "" CODE [start [, len]]
LOAD "" SCREEN$
VERIFY …                       ' same forms as LOAD
CAT                            ' list tape headers until BREAK
```

Direct-mode commands

`RUN [label]`, `LIST [proc|label]`, `NEW`, `CONT` (not after an edit), `EDIT`, `FIND "text"`, `FREE`. Any statement can also be typed in direct mode.

4.5 Functions

Group	Functions
Maths	`ABS SGN INT SQR EXP LN LOG SIN COS TAN ATN ASN ACS PI RND RND(n) MAX(a,b) MIN(a,b)`
String	`LEN LEFT$ RIGHT$ MID$ CHR$ ASC STR$ VAL INSTR(a$,b$[,start]) STRING$(n,s$) HEX$(n) UPPER$`
Input	`INKEY$`, `KEY(k$)` (true while that key is held, with no repeat, which suits games), `JOY` (Kempston)
Machine	`PEEK DPEEK IN(port) USR(addr)` (returns BC, as Sinclair does), `FRAMES`, `FREE`
Screen	`POINT(x,y) ATTR(r,c) SCREEN$(r,c)`. `ATTR` takes a text cell and works out the attribute cell under it (§6.4).
Errors	`ERR`, `ERR$`, `ERRLINE` (line ordinal), `ERRPROC$`

4.6 Error handling

`TRY` pushes a frame holding the stack pointer and the address of its `CATCH`. An error, whether raised by the ROM or by `ERROR n`, unwinds the control stack to the nearest TRY frame. Along the way it restores LOCALs and drops FOR, WHILE and PROC frames, then continues after `CATCH`. Without a TRY, execution stops and the editor opens on the offending line. The message is shown on the status bar, for example `6 Number too big, line 57 in PROC move`. BREAK (CAPS+SPACE) is error 1. It can be caught, but a second BREAK within 1 second cannot.

4.7 Sample programs

Bouncing ball (64-column text with UDG graphics)

```
' Bouncing ball demo for OpticOS
DEF PROC init
  BORDER 0: PAPER 0: INK 7: CLS
  UDG "A", $3C,$7E,$FF,$FF,$FF,$FF,$7E,$3C
  x% = 10: y% = 5: dx% = 1: dy% = 1
ENDPROC

DEF PROC move
  PRINT AT y%, x%; "  ";
  x% = x% + dx%: y% = y% + dy%
  IF x% <= 0 OR x% >= 62 THEN dx% = -dx%: BEEP 0.01, 24
  IF y% <= 0 OR y% >= 22 THEN dy% = -dy%: BEEP 0.01, 12
  PRINT AT y%, x%; INK 6; CHR$(144);
ENDPROC

init
REPEAT
  move
  WAIT 1
UNTIL KEY(" ")
PRINT AT 23, 0; "Bye!"
```

The UDG takes two text columns, so x% steps through 0–62.

Guess the number, with error handling and a recursive function

```
DEF FN fact(n)
  IF n <= 1 THEN RETURN 1
  RETURN n * fact(n - 1)
ENDFN

secret% = RND(100)
tries% = 0
PRINT "I'm thinking of a number 1-100.  (10! = "; fact(10); ")"
REPEAT
  TRY
    INPUT "Guess"; g%
  CATCH
    PRINT "Numbers only, please ("; ERR$; ")"
    g% = 0
  ENDTRY
  tries% = tries% + 1
  CASE SGN(g% - secret%) OF
    WHEN -1: PRINT g%; " is too low"
    WHEN  1: PRINT g%; " is too high"
    OTHERWISE: PRINT "Got it in "; tries%; " tries"
  ENDCASE
UNTIL g% = secret%
```

---

5. Program storage

5.1 Line format

```
+-----+----------------------------+------+-----+
| len | tokens and operands …      | $0D  | len |
+-----+----------------------------+------+-----+
```

- `len` (1 byte, 3–255) appears at **both ends** of the line. Walking forward is `ptr += len` and walking backward is `ptr -= (ptr-1)^`. Both are O(1), so the editor never needs line numbers. The overhead is 3 bytes per line, against Sinclair's 5 (number, length and ENTER).
- The program ends with a `$00` sentinel.

- **Indentation is not stored.** The tokeniser strips leading spaces. LIST and the editor recompute indentation from the block keywords, at 2 spaces per level, while they display. The program always looks tidy and no bytes are wasted.
- Spaces inside a line are kept only where they are needed to separate words, so that LIST shows a canonical spacing.

5.2 Token encoding

Byte(s)	Meaning
`$80–$FF`	Keyword tokens (one byte, 128 slots). Statements come first, so they can be dispatched by `token-$80`.
`$01 t h n name…`	Identifier: `t` = type (float, int, string, or array of each), `h` = 6-bit hash, `n` = length, then the lower-case name
`$02 lo hi`	16-bit integer literal
`$03 lo hi`	Hex literal (LIST shows it as `$…`)
`$04 lo hi`	Binary literal
`$05 e m m m m`	Float literal (5 bytes)
`$06 n chars…`	String literal
`$07 lo hi`	**Link slot**: a forward or back reference filled in by the prepass (§7.2)
`$20–$7E`	Punctuation and operators as ASCII. Two-character operators (<=, >=, <>, <<, >>) have their own tokens.

Link slots follow `IF` (block form), `ELSEIF`, `ELSE`, `WHILE`, `WEND`, `REPEAT`, `CASE`, `WHEN`, `TRY`, `CATCH`, `DEF PROC` and `DEF FN`. Each is 3 bytes, and the prepass patches it with the address of the matching keyword. Identifiers and literals are self-describing, so the detokeniser rebuilds the exact source text. The only change is that numbers come back normalised, so `1.50` lists as `1.5`.

5.3 How the editor addresses lines

The editor keeps a **(pointer, ordinal)** pair for the top line on screen and for the cursor line. Moving up or down one line is a single `len` step and ±1 on the ordinal. Jumping to an ordinal (from an error, `FIND` or `LIST name`) walks from the nearest known pair. Ordinals exist only for display. They are never stored and never used in GOTO.

5.4 RAM layout (48K)

```
$4000-$57FF  bitmap            $5800-$5AFF  attributes
$5B00-$5BFF  edit-line buffer and tokeniser workspace (free for machine code once BASIC has stopped)
$5C00-$5CB5  system variables (compatible subset at Sinclair addresses, §9)
$5CB6-$5CCA  channel table (K, S, R; Sinclair layout, so 23739 = channel S output, low byte)
$5CCB-$5D4F  OpticOS extended system variables: 32 hash heads (64), A%-Z% (52), editor state (16) …
$5D50        PROG     -> program lines … $00
             VARS     -> variable records (grow up)
             (free)
             STRHEAP  <- string heap (grows down from STKBOT)
             STKBOT   -> control stack and machine stack (grow down from RAMTOP)
RAMTOP (default $FF57) +1 -> UDGs (21 x 8 = 168 bytes) to P_RAMT $FFFF
```

The legacy shim loads a Sinclair program at `$5CCB` over the extended variables. That is safe because the shim only uses the compatible variables. Returning to OpticOS after legacy mode triggers a warm reset.

---

6. The editor and the 64-column display

6.1 The 4-pixel font (default)

- **Format.** 96 characters (32–127), each 4 pixels wide by 8 rows. **Two glyphs are packed per byte row**: characters 32+2k and 33+2k share 8 bytes, with the even code in the high nibble and the odd code in the low nibble. That makes 48 pairs × 8 bytes = **384 bytes**, at `$3E80`–`$3FFF`, aligned so that a pair's 8 bytes never cross a 256-byte page (so `INC E` steps through them).
- **Design grid.** Glyphs are 3 pixels wide with a fourth blank column, so neighbours have a gap. Row 0 is blank (line spacing), capitals and digits use rows 1–5, and descenders use rows 6–7. Lower case uses a 4-row x-height. The artwork is original and drawn by us in `font4.txt` (ASCII art), in the spirit of Tasword Two, not copied from it (§1.3).
- **The copyright sign.** Code 127 is the **editor cursor/wrap block**, not ©. A © is illegible at 3 pixels wide, so **the font has no © glyph** and the banner uses ASCII `(c)`.
- **Block graphics 128–143** are generated in code from the bits of the code. Each quadrant is 2 pixels × 4 rows. No table is needed.
- **UDGs 144–164** are 8×8. In MODE 64 a UDG fills one whole screen byte, which is **2 text columns**. It is printed at the next even column and advances 2 columns, so it always lines up with one attribute cell.

6.2 The fast print routine

The screen byte is `col >> 1`. An even column means the high nibble, so the destination mask is M = `$F0`, and an odd column gives M = `$0F`. The glyph nibble sits on the side given by `(code-32) & 1`. If that side differs from the column parity, the nibble is swapped with 4 × `RRCA`. A masked merge then writes the nibble without disturbing the other character:

```
; DE -> 8 font bytes of the pair, HL -> top screen byte, C = M, B = 8
row:  ld   a,(de)      ;  7
      ; rrca x4        ; 16  (only when parity differs; separate loop copy)
      xor  (hl)        ;  7
      and  c           ;  4
      xor  (hl)        ;  7  -> new nibble in M, old screen bits elsewhere
      ld   (hl),a      ;  7
      inc  e           ;  4
      inc  h           ;  4
      djnz row         ; 13
```

That is 53 T per row (69 with the swap), so **424–552 T for the 8 rows**. Setup adds about 280 T: channel dispatch, the screen address from row and column, the font address, the attribute write and advancing the cursor. **Total: about 700–830 T per character**, or about 750 T on average. A full 64×24 screen of 1,536 characters takes about 1.15 M T, which is about **16 frames (0.33 s)**.

- **Pair fast path.** When two characters start at an even column with plain attributes (the normal case for LIST, editor redraw and string PRINT), both nibbles are combined and the whole byte is stored with no masking: `ld a,(de1) / and $F0 / ld c,a / ld a,(de2) / and $0F / or c / ld (hl),a`, with the swaps as needed. This costs about 75 T per row, or about **450 T per character** including setup, and an editor redraw takes about 0.2 s.
- **INVERSE / OVER** use a slower general path. `CPL` before the merge gives inverse. OVER replaces the merge with `and c / xor (hl)`.
- **Scrolling** moves 23 text rows up one, exactly as in 32-column mode: 5,888 bytes by `LDI` runs, about 110–125 k T, which is under 2 frames.

6.3 MODE 32

`MODE 32` switches the print routine to a whole-byte path.

- If `CHARS` holds its default value, glyphs are the 4-pixel font with each pixel doubled horizontally, using a 16-byte nibble-to-byte table. That is chunky but readable, and it costs no font storage.
- If a program has pointed `CHARS` at an 8×8 font in RAM, as games and the legacy shim might, that font is used directly.

The cost is about **110 bytes**: the table plus the extra paths. A full 8×8 ROM font at `$3D00` would look better in MODE 32 and restore `$3D00` compatibility, but at **+768 bytes** it would leave only about 240 bytes of margin, so it is not included for now. It may be revisited in Phase 5 if the margin allows (§12).

6.4 Attributes at 64 columns: colour-clash policy

One attribute byte covers an 8×8 cell, which is **two 64-column characters** (columns 2k and 2k+1).

- **Last writer wins.** Printing any character writes ATTR_T into its cell, so it also recolours its neighbour. Colour 8 (transparent) and `OVER` keep their Sinclair meanings, so `INK 8`/`PAPER 8` leave the cell's colours alone.
- **Guidance**, shown in the manual and in the "Colour" example: change colours on even columns. `PRINT AT r,c; INK n;` with an odd `c` recolours column `c−1` too.
- **The editor is monochrome.** It uses one paper/ink pair, so the text never clashes. Colour is used only on the status bar (row 23, a whole row) and for whole-line highlights, such as the error line in red paper, which never clash because they cover full rows.
- **The cursor** is drawn by XOR-ing a 4×8 nibble block, not by FLASH, because FLASH would blink both characters in the cell. The ISR toggles it every 16 frames.
- `ATTR(r,c)` returns the attribute of cell `(r, c>>1)` in MODE 64.

6.5 Editor layout (64×24)

```
row  0-22 : program text, 23 rows x 64 columns, auto-indented (2 spaces per level)
row 23    : status bar (inverse):
            EDIT  L  ln 57/212  col 17  PROC move           FREE 31204
```

- Long lines (up to 255 tokenised bytes, about 300 characters listed) **wrap**. Code 127 appears in column 63 as a wrap mark, and the continuation rows are indented by the line's indent + 4.
- **Console screen.** Direct mode and running programs use a separate 64×24 scrolling console. The editor screen is redrawn from the program when you go back to it, so no shadow buffer is needed. `CAPS+1` (EDIT) switches between the two. With an empty program, boot lands in the console.

6.6 Typing and keys

Keywords are typed **letter by letter**, which is the recommendation. The tokeniser recognises them when you leave a line. The keyword legends on the keys are ignored, which frees the whole keyboard for typing.

Key	Action
Letters and digits	Lower case. **C** mode (caps lock) gives upper case.
SYMBOL SHIFT + key	The red symbol printed on the key (", =, +, <, ; ...), as on Sinclair
CAPS + SYMBOL, then a key	**E (extended) mode**, one shot: `[ ] { } \ | ~` on SS-style keys Y U F G D S A, and the editor functions below
CAPS+5/6/7/8	Cursor left, down, up, right
CAPS+0	Delete left. In E mode, delete right.
CAPS+2	Caps lock
CAPS+3 / CAPS+4	Page up / page down (23 rows)
CAPS+1 (EDIT)	Switch between the editor and the console

Key	Action
CAPS+9	Graphics mode (block graphics and UDGs)
ENTER	Split the line, or add a new line below
E + 5 / E + 8	Start / end of line
E + 6 / E + 7	End / start of program
E + D	Delete the line (into a one-line clipboard)
E + I	Insert the clipboard line above the cursor
E + F / E + N	Find: prompt on the status bar / find next
E + J	Join with the next line
E + R	RUN (go to the console and run)
CAPS+SPACE	BREAK

Insert and delete. The current line is detokenised into the edit buffer at `$5B00`, where editing is plain text. When the cursor leaves the line, or ENTER is pressed, the buffer is re-tokenised and swapped into the program with one block move (`LDIR`/`LDDR`) of everything above it. Any edit sets `DIRTY`, which forces a prepass and a variable clear on the next RUN and disables CONT. The tokeniser always accepts the line. Syntax and block errors come from the prepass or at run time, and jump straight to the line.

Direct commands are typed on the console: `RUN`, `LIST`, `NEW`, `SAVE`, `LOAD`, `VERIFY`, `CAT`, `CONT`, `FIND`, `FREE`, or any statement.

6.7 Boot screen

The cold-start screen (layout agreed with Nathan, 13 November 2026) is:

Row	Content
0	**Colour bar.** Attribute cells only, with no pixels set. There are 16 swatches, each 2 attribute cells wide, across all 32 columns: black, blue, red, magenta, green, cyan, yellow and white, first normal and then BRIGHT. Bright black is included so the layout stays even. Each cell's ink equals its paper.
1	Blank
2	`OpticOS (c) 2026 Optica Technology`, white on black
3	`OpticBASIC v0.1`, the name and version of the BASIC
4	Blank
5	The prompt `]`, with the cursor at column 2

Rows 1–23 are white ink on black paper, and the border is black. **`]` followed by a space is the OpticOS direct-mode prompt everywhere.** It appears at boot and again whenever a command finishes or an error is reported. The prompt and its space use two of the 64 columns, so a console line holds up to 61 characters plus the cursor. The colour bar is an ordinary screen row, so it scrolls away with the console text. The mock-up is `assets/opticos_boot.png`, and the rig's boot test compares the ROM's screen with it pixel for pixel.

---

7. Interpreter design

7.1 The tokeniser

The tokeniser makes a single pass over the edit buffer. At each word boundary it looks the word up in the keyword table. The table is sorted and grouped by first letter, with a 26-entry index of first-letter offsets (52 bytes), and each keyword ends with bit 7 set. A match must be followed by a non-identifier character, and `%`, `$` or `(` after a keyword are allowed by context. Words that don't match become

identifiers: the tokeniser computes their hash and stores the type, hash, length and name. Numbers are converted to binary with the maths package's string-to-number routine. The detokeniser does the reverse, plus indentation, and serves LIST, the editor and error display.

7.2 The prepass (resolver)

The prepass runs at RUN, or at the first CONT after a load or edit. It takes about 30–60 ms per KB of program. In one linear pass it:

1. Records every `DEF PROC`, `DEF FN` and `LABEL` in a **symbol table** in a reserved RAM area just above the program: hash, name pointer and line address, 5 bytes each. It reports duplicates.
2. Matches blocks with a small temporary stack and **patches the link slots**: - `IF` → its `ELSEIF`, `ELSE` or `ENDIF` - `ELSE` → `ENDIF` - `WHILE` ↔ `WEND` - `REPEAT` ← `UNTIL` - `CASE` → each `WHEN` → `ENDCASE` - `TRY` → `CATCH` → `ENDTRY` - `DEF` → its end, so that execution can skip over definitions it falls into
3. Reports structural errors before anything runs, for example "ENDIF without IF, line 40".

At run time, a false `IF`, an exiting `WHILE` or a skipped `DEF` costs one pointer load. There are no scans.

7.3 Statement dispatch and the control stack

The interpreter's registers are HL = text pointer (`CH_ADD`) and SP = the machine stack, which is also the control stack. Each statement token is looked up in a 2-byte jump table. Control frames are pushed on the Z80 stack with a type byte:

Frame	Contents	Bytes
FOR (float)	type, variable pointer, limit (5), step (5), loop pointer	15
FOR (integer fast)	type, variable pointer, limit (2), step (2), loop pointer	9
WHILE / REPEAT	type, loop pointer	3
PROC / FN	type, return pointer, saved-locals count, saved ERR frame	6
LOCAL entry	variable pointer, old value (2, 5 or 3 bytes)	4–7
TRY	type, CATCH address, saved SP	5
CASE	type, ENDCASE address, selector value	3–8

`ERR_SP` points at the current error frame. RST 8 unwinds to that frame. `NEXT`, `WEND` and `ENDPROC` check the frame type. When they find the wrong one, they report "NEXT without FOR" and similar errors. Before each push, a stack-overflow check compares SP against `STRHEAP` plus a 64-byte guard.

7.4 Variable lookup

The type byte and 6-bit hash are stored in the identifier token. There are 32 chain heads, indexed by hash and type. Each variable record holds: next pointer (2), name length (1), name, value (2, 5 or 3 bytes, or an array header).

Lookup: take the head, walk the chain, and compare the length and then the name with a short `CPI`-style loop. A typical program has under 50 variables, so chains average fewer than 2 entries, and a lookup costs about 150–250 T. `A%–Z%` skip lookup entirely at about 40 T. Creating a variable appends at `VARS` end, which sits below the heap, so the gap shrinks.

A later optimisation, kept out of the budget, is an inline cache: a 2-byte pointer slot tagged with a RUN generation counter. It would cost 3 bytes per identifier use, which is why it isn't in the plan.

7.5 The integer fast path

The expression evaluator is precedence-climbing. Values live on a small value stack in workspace as 5-byte entries. Integers of every kind are held in **small-integer form** (`00 sign lo hi 00`).

For `+ - * < = …`, `DIV`, `MOD`, `AND`, `OR` and shifts, if both operands are small integers, 16-bit Z80 code runs and checks for overflow. Only on overflow does it promote both operands to float and retry. `FOR` with integer start, limit and step uses the 9-byte frame and 16-bit compare. Literals are pre-converted, so there is no ASCII parsing at run time. Between them, these are where the speed comes from.

7.6 Strings and garbage collection

- The heap grows **down** from `STKBOT`. Each heap string is `[back-pointer 2][length 1][chars]`, and the back-pointer points at the descriptor that owns it.
- Temporaries made during expression evaluation go on the heap with back-pointer 0. They are freed at the end of each statement by resetting the heap mark (no GC).
- **Compaction** runs when an allocation fails. It is a sliding pass from the top of the heap down:
- If a block's back-pointer still points at a descriptor that refers to this block, the block is kept: it slides up and the descriptor is fixed up.
- Otherwise the block is dropped.
- The pass is O(heap) and needs no mark phase or extra memory, at about 300 bytes of code.
- The worst case is about 25 ms per 10 KB of heap.

7.7 Maths package

- **Format.** Sinclair-compatible 5-byte layout. That keeps `STACK-BC`-style interoperability possible, and the small-integer form provides the fast path.
- **Core.** Add, subtract, multiply (32×32 shift-and-add), divide (restoring), normalise and round, compare, and int↔float conversion.
- **Transcendentals.** Range reduction plus short Chebyshev or minimax series, driven by a compact **internal byte-code** (our own opcodes) that runs coefficient tables. That is far smaller than open-coding each function.

Function	Method
SIN, COS	Reduce to ±π/4, 6 terms
ATN	Reduce to [0, 1], 8 terms. ASN and ACS are derived from it.
LN	Mantissa in [0.5, 1], 7 terms
EXP	2^n × series
SQR	Newton iteration, 4–5 steps
^	EXP(LN) with an integer fast path for small exponents

- **Accuracy target**: an error of 1 unit in the last place or less on the core operations, and 2 or less on the transcendentals, checked against Python `mpmath`.

---

8. 16K ROM memory map and budget

8.1 Fixed points in the ROM

```
$0000-$006F  RST vectors, $0038 ISR entry, $0066 NMI         (fixed)
$03B5        BEEPER          $04C2 SA-BYTES   $0556 LD-BYTES
$05E3/$05E7  LD-EDGE-2/1     $096F RET        $0D6B CLS   $0DAF CL-ALL
$1601        CHAN-OPEN       $203C PR-STRING  $22AA PIXEL-ADD
$3E80-$3FFF  4-pixel font, 384 bytes                          (fixed)
everything else: modules packed into the gaps by the link script (first-fit, largest first)
```

8.2 Budget

#	Module	v0.2 budget	Built after phase 3	v0.3 budget
1	Restarts, NMI, reset, RAM test, system-variable and channel init	380	292	300
2	Boot banner plus the call to print it	40	61	65
3	Interrupt routine, keyboard scan, debounce and repeat, key tables	520	355	420
4	64-column print, control codes, AT/TAB, colours, scroll, cursor (INVERSE/OVER to come)	1,000	578	800
5	MODE 32	110	–	110
6	Block-graphics generator and 2-column UDG print	60	–	60
7	**4-pixel font, 96 glyphs, 2 per byte**	384	384	384
8	Tape low level: SA-BYTES, LD-BYTES, edge routines	380	310	330
9	Tape commands: SAVE/LOAD/VERIFY/CAT, headers (only LOAD "" so far)	520	115	450
10	Sound: BEEPER, BEEP, SOUND	160	82	160
11	Graphics: PLOT, DRAW, CIRCLE, POINT, ATTR, SCREEN$	620	–	620
12	Float core	1,100	1,424	1,424
13	Number ↔ string, pr_dec / pr_hex	550	788	800
14	Transcendentals, byte-code runner, coefficient tables	950	1,320	1,320
15–17	Tokeniser, LIST auto-indent, keyword table, dispatch tables	1,500	1,569	1,600
18	Expression evaluator and functions	850	1,362	1,400
19	Interpreter core: run loop, statements, prepass, PROC/FN/LOCAL/TRY (EXIT etc. to come)	1,550	2,286	2,450
20	CASE / WHEN / OTHERWISE	150	–	150
21–22	Variables, arrays, string heap and GC, value and control stacks (INSTR etc. to come)	1,150	1,288	1,400
23	I/O and miscellaneous statements (INPUT, DATA/READ, IN/OUT, CALL/USR, KEY, JOY …)	450	–	450
24	Full-screen editor	1,400	–	1,400
25	Direct mode and console	250	221	250
26	Errors: RST 8 unwind and messages	450	490	520
27	Legacy loader shim (Sinclair BASIC subset)	650	838	850
28	Fixed-address stubs and fragmentation loss around fixed points	200	78	200
	Total	**15,374**	**13,841**	**17,913**
	Margin (of 16,384)	**1,010**	**2,543 free now**	**−1,529 (over)**

Rebalance after phase 3 (v0.3). The maths (items 12–14, +882 bytes) and the interpreter (items 15–22, +1,450 bytes) came out larger than the v0.2 estimates. As agreed, the size reduction is left to phase 5, and the budgets above now record what was built, with room kept for the features each module still lacks. The ROM built so far fits, with 2,543 bytes free (well above the 256-byte margin rule). The free space is in seven gaps between fixed addresses, the largest being 613 bytes, so later modules must be split to fit.

Warning: the full feature list no longer fits. With the v0.3 budgets the finished ROM would need 17,913 bytes, which is 1,529 over 16,384, or about **1,785 bytes over** once the 256-byte margin is kept. Phase 5 therefore has to save about 1.8 KB, by shrinking the maths and interpreter, cutting features, or both. The likely sources are: the byte-code runner used more widely in the maths (about

300–400); the evaluator and statement code tightened, for example shared argument parsing and RST-based helpers (about 400–600); a smaller legacy shim (about 200); shorter error texts made from keyword tokens (about 150); and dropping or simplifying the MODE 32, CASE or SOUND items if that is still not enough.

The 4-pixel font saves 384 bytes compared with an 8×8 set. The 64-column print logic costs about 100 bytes more than a plain 32-column printer, so the change of display gained roughly 200 bytes overall. Adding an 8×8 font at `$3D00` would bring the total to 16,142, leaving a margin of 242 (1.5%). That is too thin, so the font is left out for now (§12).

8.3 Benchmarks

- **ZX81 8K ROM.** Floating point, BASIC, display, tape and a line editor in 8,192 bytes. That suggests a richer system in 16K is realistic.
- **Sinclair 48K ROM.** 16K, including a 768-byte font and a large calculator. It is generous in its token handling, and its syntax checking is folded into the run-time code.
- **BBC BASIC II.** 16K for the language alone. The screen, keyboard, tape and line editor live in the separate 16K MOS, and about 2–3K of the language ROM goes on the inline assembler. Without the assembler that leaves about 13K for language plus maths, which is the strongest warning against over-ambitious features. OpticBASIC is smaller (no assembler, no VDU system, simpler PRINT formatting), but it also carries its own "MOS".
- **Microsoft 8K BASIC** (8080 and 6502 versions, about 8–9K with floats) shows that the core interpreter plus floats can be done in about 7K.

8.4 Risks

Risk	Likelihood	Mitigation
The maths package and editor overrun by 20–30% (typical)	High	Margin, plus the cut list below
LD-EDGE timing mismatch breaks turbo loaders	Medium	Pulse-sweep tests against the reference in the rig. A loader compatibility corpus.
Fixed addresses fragment the ROM	Medium	Link script with gap packing. Small routines are kept to fill gaps.
64-column legibility on a real TV	Medium	Font tuning in Fuse with a CRT filter. MODE 32 as a fallback.
Float accuracy	Low to medium	mpmath test vectors

Cut list (about 900 bytes recoverable): CASE (150), ASN/ACS (80), `SCREEN$()` (60), CAT (60), CIRCLE (120), editor clipboard and join (150), `HEX$/STRING$/UPPER$` (90), and slimming the legacy shim to the minimal loader set (200).

---

9. System variables

Policy. Keep the variables that machine code and loaders commonly touch at their **Sinclair addresses**, with the same meaning. Everything else in `$5C00–$5CB5` is reused for OpticOS internals, and `IY` is set to `$5C3A` whenever control passes to user code. This keeps `(IY+n)` habits harmless, and the OpticOS ISR does not rely on IY.

Address	Name	Kept?	Why
`$5C08`	LAST_K	Yes	Games read it for keys
`$5C09/0A`	REPDEL/REPPER	Yes	Commonly POKEd

Address	Name	Kept?	Why
$5C36	CHARS	Yes	Fonts in RAM. It also selects MODE 32 behaviour.
$5C3A	ERR_NR	Yes	IY base, RST 8
$5C3B	FLAGS	Bit 5 (new key) only	ISR and keyboard pollers
$5C3D	ERR_SP	Yes (OpticOS meaning: error frame)	Code that traps errors
$5C48	BORDCR	Yes	Restoring the border after a load
$5C4F/51	CHANS/CURCHL	Yes	RST 16 and CHAN-OPEN users
$5C53	PROG	Yes (OpticOS program start, or $5CCB in legacy mode)	PEEK 23635 loaders
$5C6B	DF_SZ	Yes	Lower-screen size
$5C76	SEED	Yes	RND
$5C78	FRAMES (3)	Yes	Timing in many games
$5C7B	UDG	Yes	UDG relocation
$5C7D	COORDS	Yes	PLOT position
$5C88	S_POSN	Yes	Print position, stored as 64-column values in MODE 64
$5C8D–$5C91	ATTR_P, MASK_P, ATTR_T, MASK_T, P_FLAG	Yes	Colour handling
$5CB0	NMIADD	Yes	NMI hook
$5CB2	RAMTOP	Yes	CLEAR, and loaders
$5CB4	P_RAMT	Yes	Memory size
others	KSTATE, E_LINE, WORKSP, X_PTR ...	Reused internally	Only Sinclair BASIC needs their meanings

New variables in $5CCB–$5D4F:

- MODE
- FONT4
- VARS, STRHEAP, STKBOT, SYMTAB
- DIRTY and the prepass generation counter
- editor state: top pointer and ordinal, cursor pointer and ordinal, column, clipboard
- the 32 hash heads (64 bytes)
- A%–Z% (52 bytes)

A magic byte and checksum on this area lets OpticOS detect corruption by machine code. A RET back from CALL into a damaged area causes a warm reset that keeps the program, rather than a crash.

---

10. Build and test plan

10.1 Tools on the box

- **Assembler.** pasmo 0.5.3. Install it from /workspace/debs/pasmo_0.5.3-8_amd64.deb (dpkg -x into a local prefix if it isn't on PATH; today it isn't). Use pasmo --bin opticos.asm build/opticos.rom build/opticos.sym. The Python assembler spectrum-ball/tools/zasm.py is a fallback that produces identical output.
- **Emulator.** Fuse 1.6.0 (fuse-emulator-sdl and libspectrum8 debs in /workspace/debs), driven by ultimate/miremare/tools/fusectl.py, which gives a private Xvfb, xdotool keys and screenshots. Load the ROM with: fuse-sdl --machine 48 --rom-48 /workspace/newrom/build/opticos.rom --no-traps --no-fastload file.tap Fuse's tape traps assume the Sinclair loader internals, so the

real-signal path (`--no-traps`) is the authority. A second pass with traps on records whether fast loading happens to work.

- **Headless rig.** `z80core.c` / `libz80core.so` (copies in `spectrum-ball/tools` and `miremare/tools`), driven from Python through `spectest.py` and `headless.py`. Today the rig injects game code directly and **does not implement IX/IY (DD/FD prefixes)** or EAR input. Phase 0 extends it with: 1. the DD/FD/DDCB/FDCB opcodes (needed for LD-BYTES, whose contract is IX-based) 2. a ROM-boot mode from `$0000` with a 16K ROM write-protect 3. an EAR input pulse stream generated from a TAP (the standard timings) and a MIC/EAR output recorder that turns pulses back into TAP 4. memory-contention timing that is accurate enough for edge-loop tests.

10.2 Build checks (`make check`)

1. The image is exactly 16,384 bytes, and the fixed symbols sit at their addresses (checked from the symbol file).
2. `similarity_check.py` passes against `48.rom` (§1.3). The ROM is never copied into `build/`, and `.gitignore` covers `roms/`, `*.rom` from outside and `build/`.
3. A size report per module against the budget table. The build fails if margin drops below 256 bytes.

10.3 Test suites

Suite	How it works
Boot	Rig: the banner `OpticOS (c) 2026 Optica Technology` appears at row 0, read back with a Python decoder that converts screen bitmaps to text using our font. RAM test passes for 16K and 48K.
Keyboard	Inject matrix states. Check LAST_K, repeat and E mode.
Print	Pixel-exact checks at odd and even columns, pair path, INVERSE/OVER, attribute-cell policy, scrolling. T-state counts per character recorded (target ≤ 850 T masked, ≤ 500 T pair).
Maths	About 5,000 vectors against mpmath, with error in units of the last place. Integer fast path and overflow promotion.
Tokeniser	Round trip: source → tokens → LIST is identical after normalisation, across a corpus.
Interpreter	About 150 small `.obas` programs with expected console output, including the §4.7 samples, recursion, GC stress and every error message.
Editor	Scripted key sequences, then compare the program and screen.
Performance	PCW benchmarks BM1–BM8 under OpticOS and under the Sinclair reference in the rig. Target ≥ 3× on BM1–BM3.
Tape: save	SAVE → recorded pulses → TAP → byte-compare and parse. The original ROM in Fuse then loads OpticOS-saved CODE blocks.
Compatibility	`LOAD ""` with real-signal loading in Fuse (`--no-traps`): `/workspace/spectrum-ball/bounceball.tap`, `/workspace/rel/miremare.tap`, `miremare_demo.tap` and `miremare_sfx.tap`. Pass means the game reaches its title screen and responds to keys (screenshots compared with the earlier runs under the Sinclair ROM). Extend later with any TAPs Nathan supplies privately, which are never committed.
Edge timing	Sweep pulse lengths and compare the B returned by `$05E7` with the reference ROM, tolerance ±1.

---

11. Phased plan

A "session" means one long working block, as with Mire Mare.

Phase	Content	Deliverables	Effort
0. Rig	Add IX/IY, ROM boot, EAR in and out, and the screen-to-text decoder to the headless core. Build skeleton, link script, checks.	`tools/rig/`, `make check`, an empty 16K image	1 session

Phase	Content	Deliverables	Effort
1. Boot, screen, keyboard, tape	Reset and RAM test, ISR, keyboard, 4-pixel font artwork, 64-column print and scroll, MODE 32, channels and RST 16, BEEPER, SA/LD-BYTES at fixed addresses, the stubs, **the legacy shim**, and a minimal console.	A ROM that boots to the banner in Fuse. Bounce Ball and Mire Mare load and play via `LOAD ""` with no traps. Font sheet PNG.	3 sessions
2. Maths	Float core, conversions, byte-code runner, transcendentals, RND	Maths library plus the mpmath test report	2 sessions
3. Interpreter	Tokeniser and detokeniser, prepass, evaluator, statements, PROC/FN/LOCAL, loops, TRY, strings and GC, arrays, graphics, sound, tape commands, console-based direct mode	Sample programs run. Benchmark report against Sinclair.	3–4 sessions
4. Editor	Full-screen 64×24 editor, keys, wrap, find, clipboard, status bar, error jump	Editor test suite. Demo GIF.	2 sessions
5. Polish	CASE, size squeeze, a possible revisit of the 8×8 font if the margin allows, user manual (PDF, in the Mire Mare style), example tape, a check on a real TV via Nathan	`opticos.rom` v1.0, manual, examples. The repo is **OpticOS**, created only with Nathan's go-ahead.	1–2 sessions

The total is about **12–14 sessions**, which is larger than Mire Mare. Phase 1 is a useful stopping point on its own: it proves the hardware layer and tape compatibility before any BASIC exists.

---

12. Decisions (agreed 13 Nov 2026)

Nathan agreed all six recommendations on 13 November 2026.

1. **No 8×8 font at `$3D00` for now.** Ship the 4-pixel font only and get MODE 32 by pixel doubling. Revisit in Phase 5 if the margin allows. Without it, games that draw text from the ROM font show garbage.
2. **Include the legacy loader shim (about 650 bytes).** Without it, almost no commercial tape, including Bounce Ball and Mire Mare, loads with `LOAD ""`.
3. **Error handling is `TRY/CATCH/ENDTRY` plus `ERROR n`.** It is structured, it fits the no-line-number style, and it costs about the same as `ON ERROR`.
4. **Procedures are called by bare name** (`move`, not `PROC move`). They look modern, the prepass resolves them, and the cost is a small lookup on the first identifier of a statement.
5. **Do not emulate the RST `$28` calculator.** It would cost 600 bytes or more, and few games use it. RST `$28` raises a clean error instead.
6. **`LOCAL` is dynamic, in the BBC style**, not true lexical scope. Lexical scope would need per-PROC symbol tables, at about 300 bytes plus a slower lookup.

End of specification v0.2.